```
import { q, one } from '../db.js';
import { resolve, resolveAll } from './providers.js';
import { listCapabilities, resolveDisposition } from './policy.js';
import { request } from './executor.js';
import { route } from './router.js';

// A harness gets a goal, not a database. What it can reach is exactly what the
// acting person could reach by hand: the same grants, the same approvals, the
// same receipts. A harness cannot widen its own permissions.
export async function runHarness({ ctx, goal, harnessKey = null }) {
  const providers = await resolveAll({ slot: 'harness', businessId: ctx.businessI
  if (providers.length === 0) throw new Error('no harness provider bound');

  const chosen = harnessKey
    ? providers.find(p => p.manifest.key === harnessKey)
    : providers[0];
  if (!chosen) throw new Error(`harness "${harnessKey}" is not bound for this bus

  const run = await one(
    `insert into harness_run (business_id, harness_key, goal) values ($1,$2,$3) r
    [ctx.businessId, chosen.manifest.key, goal]
  );

  // Only capabilities this actor is actually allowed to invoke are offered.
  const all = listCapabilities();
  const allowed = [];
  const withheld = [];
  for (const cap of all) {
    // Anything that edits an outside app through the device is a script.
    // A model may read those apps; it may not write to them.
    if (cap.agentSafe === false) { withheld.push({ key: cap.key, why: 'script-onl
    const d = await resolveDisposition({
      businessId: ctx.businessId, membership: ctx.membership,
      capability: cap.key, system: ctx.system === true,
    });
    if (d === 'never') { withheld.push({ key: cap.key, why: 'not granted' }); con
    allowed.push(cap);
  }

  const invoke = ({ capability, input }) => request({ ctx, capability, input });
  const think = (task) => route({ ctx, task });

  let out;
  try {
```

```
    out = await chosen.module.run({ ctx, goal, capabilities: allowed, invoke, thi
    out.withheld = withheld;
  } catch (e) {
    out = { state: 'failed', reason: e.message, steps: [], withheld };
  }

  return one(
    `update harness_run set state = $1, steps = $2::jsonb, result = $3::jsonb, fi
      where id = $4 returning *`,
    [out.state, JSON.stringify(out.steps ?? []), JSON.stringify(out), run.id]
  );
}

export async function runs(businessId, limit = 50) {
  return q(
    `select harness_key, goal, state, finished_at from harness_run
      where business_id = $1 order by created_at desc limit ${Number(limit)}`, [b
  );
}
```